

Implementation experience on trying to implement concurrent data and control structures with executors

Doc. No.: P1257R0

Date: 2018-10-16

Reply to: Detlef Vollmann, dv@vollmann.ch

Audience: SG1, (LEWG)

Abstract

At the meeting 2016 in Oulu I presented a number of challenges / requirements on executors from a concurrency perspective. This paper reports the experience when I tried to implement some of the examples in that presentation based on the proposal in [P0443R4](#).

This paper concludes with a list of unsolved problems that an executor proposal must address before it would be useful for concurrency use cases.

Introduction

In my presentation in Oulu I presented several examples that are typical concurrent data and control structures but that couldn't be implemented based on the proposals of that time.

In the meantime we got [P0443](#). So I tried to implement the examples in that presentation based on P0443 and the respective demo implementation available on [GitHub](#). It was surprisingly hard and took much more time than I expected. When I started my implementation [P0443R4](#) was the current version so my implementation is still based on that version, but I don't think that any of the changes since then make a major difference.

For anybody interested in the full code, it's available at <https://gitlab.com/cppz/executor-examples/blob/master/doc/code-details.md>. It probably has bugs, but compiles and passes some tests.

Too keep this paper short it only presents the problems and the chosen solutions in an abstract way. For more details please see <https://gitlab.com/cppz/executor-examples/blob/master/doc/code-details.md>.

Basic Data Concentrator

One of the examples for concurrent control structures was a data concentrator. It consists of three tasks, two of them reading input (typically from an external device) and one task to potentially run some filter and store the data (or write it to another external device). Such a use case is not universal enough to standardize a respective control structure but is still common enough to provide it as library.

Based on a queue as proposed in [P0260](#) and a latch as proposed in [P0666](#), it's not hard to come up with an implementation for concentrator that just creates a queue for communication, a latch for the providing tasks to signal when finished, and then starts tasks on a given executor with a small wrapper that calls the given functions and handles the queue interface:

```
```c++ template class Concentrator { public: Concentrator(IN1 producer1, IN2 producer2, OUT consumer, size_t bufSize);

template <class Ex> void run(Ex exec);

private:

 IN1 p1;
 IN2 p2;
 OUT c;
 buffer_queue<T> q;
 latch inDone;

}; ```
```

(This could be even simpler by just providing a single `run()` function or at least by giving deduction guides to the constructor.)

To use the concentrator just create one and start it on an executor:

```
```c++ auto prod1 = [] () { /* ... */ }; auto prod2 = [] () { / ... / }; auto consume = [] (int v) { / ... */ };

Concentrator dataConcentrator(prod1, prod2, consume, 1);

static_thread_pool pool{5};
auto exec = pool.executor();
dataConcentrator.run(exec);

...`
```

So basic concurrency works.

Real-Time

In the real world one of the input devices might be time critical and the task reading from it should have a higher priority.

For this we need an executor that provides priorities. This can be done by providing an execution context `RTThreadContext` and an executor (that only supports oneway) based on it.

But how do we tell our Concentrator that one of the tasks needs a different executor? [P0113](#) (and the Networking TS) have `wrap()` and `get_associated_executor()` for this. P0443 doesn't provide it, but the `require()` mechanism can be used for this.

The idea is to ask each task for its executor and then run it on this executor:

```
c++      template <class Ex> void run(Ex exec)      {          // ...          auto exIn1A = require(exec, p1);          auto exIn1B = require(exIr
```

With a generic `require()` an executor can be associated with a task by providing a respective overload:

```
c++ RTThreadContext RTContext; template<class Ex> RTExecutor require(Ex ex, decltype(prod1) &) {      return RTContext.get_executor(80); }
```

This specific mechanism doesn't work anymore in recent versions of P0443 due to changes in the `require` specification. But it was pointed out that it's still doable.

Whether using `require()` for this is a good idea is a different question. I would prefer an approach along the lines of `wrap()/get_associated_executor()`, but as long as such an interface doesn't exist, generic control structures need to use `require()` to get the executor required by a task.

Boost Blocking

The data concentrator works fine in a classic concurrent world. But it breaks down if the executor(s) given to it don't provide concurrent forward progress and can't be synchronized with `std::mutex` and `std::condition_variable`.

But one of the goals of the executor proposal is to provide a common interface for all kinds of executors, even if they provide only weakly parallel forward progress.

And something like a data concentrator should be able to run in a boost blocking environment.

To try this I implemented an executor `TGExecutor` that completely relies on boost blocking. It's based on stackful coroutines and implemented with `boost::context`.

To make the implementation easier `TGExecutor` is based on an execution context `TPool` and blocking is only defined inside the same pool or from the outside as entry point. In any given `TPool` only one task can make progress. This allows the implementation to identify the current task.

Tasks that are not blocked on aren't executed. So sometimes tasks are not completed. The destructor of such a task does not complete the task, but it will call the destructors of the local objects of that task (stack unwinding).

[P0073R2](#) proposed an event class with a `signal()` and `block()` interface. `TGExecutor` provides a class `TGEvent` with such functions as members.

Queue

The implementation of the data concentrator is based on a concurrent queue. We need a concurrent queue that can work with boost blocking and can work with "normal" concurrent threads as well. Such a queue would be templated on the executors involved and was outlined in my presentation in Oulu. My implementation now is simpler as it assumes at least compatible executors on all sides.

The standard (non-lock-free) implementation of a queue is based on condition variables and mutexes. I decided to keep such an implementation. For concurrent threads C++ provides mutexes and condition variables. For thread based executors we want to use these, so for other executors the interface should be as similar as possible.

Mutex

In a not really concurrent boost blocking world a mutex doesn't need to provide synchronization but just (possibly) boosting and still mutual exclusion.

To keep the implementation simple I support only a single task waiting for a blocked mutex. But by keeping an internal stack multiple waiters would be possible as well.

My mutex still provides the usual interface but on `lock()` it needs to store a reference to the current task so it can be boosted if another task tries to lock it. For access to the current task it needs access to our `TPool`. So the constructor needs somehow a reference to `TPool`.

Condition Variable

Implementing a condition variable in a boost blocking context is generally easy: in `notify_*` we just boost block the task(s) currently waiting, and in `wait()` we just block. The question is to block on who? How do we know who's notifying a condition variable and who we need to boost if we're blocking on a condition variable? The only way is to tell the condition variable explicitly.

This means that we not only need the standard interface `wait()`, `notify_one()` and `notify_all()`, but also a register interface. In my implementation I provide `attach(Handle &task)` and `detach()`. I return a handle from `attach` to allow for `detach`.

Again we need a somehow the `TPool` in the constructor. And I set a hard limit on attachers to avoid dynamic memory allocation.

One interesting detail is `notify_all()`, which just calls `notify_one()`. Currently I only support multiple notifiers, but not multiple waiters. For multiple waiters we wouldn't need another register interface, but like for mutex a stack that we fill on `block()` calls.

Handle

As mentioned before a boost blocking condition variable needs a register interface that registers the notifying task. We could require that the register function is called from the task that notifies and rely on the internal knowledge of the current task.

But this restricts the usability as sometimes the synchronization mechanism is setup from the outside. So I decided to return from `twoway_execute` a handle to the task. This handle can be used to get something like a future to wait on, a condition variable that's already attached, a mutex (just to put things together) or to attach to a condition variable. I didn't call this handle `future` as this would be misleading.

I don't return such a handle from `execute()` as for fire-and-forget tasks the handle would still be overhead.

Execution context

Synchronization primitives are probably not executor specific but specific to the execution context.

So I decided to provide an interface in `TPool` to get mutexes and condition variables:

```
c++ class TPool { public:      // ...      SyncMutex<TEvent> getMutex() const;      SyncCondVar<TEvent> getEmptyCondVar() const; }; This
provides condition variables with no attached tasks.
```

Queue again

Based on our mutex and condition variable a queue implementation can be started.

The attach / detach requirements of the condition variable make a general queue interface where everybody can push or pop not feasible. Instead we go with one part of P0260 that proposes front and back ends. Our queue implementation requires a separate end for each pushing or popping task.

The base implementation for the queue now needs a template parameter for the executor type. A better option than the executor type would probably be the type of the execution context. I only have a single executor type parameter even though we can have different executors pushing or popping to/from the queue. But the (current) requirement is that the synchronization primitives can at least cooperate if they are not the same.

TQueueTrunk doesn't provide any public interface except constructors. The trunk also implements the `close()`, `push()` and `pop()` methods, but as private, and it declares the end types as friends.

`push()` and `pop()` are essentially the same as for "standard" queue implementations.

`close()` is different: I require that only the back ends (that push) can close a queue and that all of them must close it before it's really considered closed.

Queue ends can be created from a queue trunk and a handle for the task to attach.

More generic concentrator

Based on the new queue the concentrator can be rewritten to work with more kinds of executors.

The latch isn't needed anymore because closing the queue has changed.

But now separate queue ends for each task is required. But for creating the queue ends a handle to the task is required. So I first create pointers to the ends that I capture by reference for the tasks, queue the tasks with `twoway_execute()` in the respective executor and use the returned handle to finally create the queue ends and let the pointers point to them. To avoid that the pointers are dereferenced before the queue ends actually are created I use a condition variable.

Everything would be much simpler if `twoway_execute()` would give a reference to the handle of the created task to the function object started inside the task, but for now we don't have this.

This modified concentrator can now be used essentially like the previous one, but now works with more kinds of executors.

Mission accomplished, but only by inventing a lot of things that are still missing from the executor proposal.

Notes

Forward progress guarantees

I thought about the kind of executors my concentrator can work with. And it turned out that the different forward progress guarantees given in [intro.progress] don't really help:

My concentrator can clearly work on executors with concurrent forward progress (if they provide synchronization primitives with the correct interface).

Parallel forward progress generally doesn't work as many executors with a parallel forward progress guarantee can't provide the correct boost blocking semantics. For parallel executors it might be generally hard to provide boost blocking and still give full concurrent forward progress on tasks already started.

My executor based on stackful coroutines only provides weakly parallel forward progress, but is still usable for my concentrator as it provides the right kind of boost blocking.

But I don't have yet the right vocabulary to specify the concrete requirements that my concentrator has on the executor given to it.

Networking

To make the concentrator actually useful in a boost blocking world the networking library needs to work with this.

I haven't actually tried it, but a correct incorporation of the executor interface for the networking library should also work with boost blocking.

Different types of executors

I haven't explored how to implement concurrent queues that has different kind of executors pushing to or popping from it.

It's probably not possible to do this generically, but for a given pair of executors it often should be possible to provide compatible synchronization primitives.

A really interesting executor would be one that uses true concurrency as long as hardware resources are available and then starts to fallback to coroutines.

I'm not sure how to implement such an executor.

Conclusion

In general it is possible to implement generic concurrent data and control structures that can work with different types of executors. But the executor proposal from P0443 is insufficient to be useful for this.

Before it can be actually useful it needs to specify at least some requirements on synchronization primitives. These requirements need to include an `attach()/detach()` interface for condition variables and an interface to create synchronization object from an executor or an execution context (or a different approach that works with boost

blocking). And for `std::thread` we need to provide respective primitives.

An executor proposal also needs to specify the requirements on the handles returned from `twoway_execute()`.

It's important to decide what should go into executor types and what should go to execution context types. Synchronization primitives belong semantically to the execution context, but there's also the wish to have executors without execution contexts.

It probably makes sense to always call the function object given to `twoway_execute()` with a reference to the handle returned by `twoway_execute()`. Otherwise another execution function (or an overload for `twoway_execute()`) is required that calls the function object with the handle.

`get_associated_executor()` and `wrap()` from P0113 should be added to the executor proposal to avoid the (ab)use of `require()` for getting the executor a task should be executed on. Otherwise at least the changes in P0443R5 on `require()` should be reverted to make it easier to use the `required()` mechanism for this.

Finally it would be useful to provide helper constructs to avoid too much boilerplate for own executor types.